Interrupciones externas

Lo del botón de la semana pasada era sondeo. El loop revisaba digitalRead cada vuelta, o sea está preguntando si ya tocaron.

Si tiene un delay de 2 segundos, en ese rato puede presionar alguien y el programa ni se entera. El toque fue antes de que vuelva a leer.

Interrupción: el GPIO avisa al procesador cuando cambia el pin. No es necesario estar revisando desde loop.

La CPU deja un momento lo que estaba haciendo, se ejecuta la ISR y luego continúa. El timer también hacía interrupciones, pero ese está dentro del chip.

ISR = función que atiende el aviso.
No es igual que saludar(); porque no la estoy llamando desde mi código.

Nivel y flanco

Nivel: se mantiene en 1 o en 0
Flanco: cuando cambia
RISING 0 → 1
FALLING 1 → 0
CHANGE cuando cambia a cualquiera de los dos.

Con INPUT_PULLUP el botón suelto marca 1. Al presionarlo pasa a 0 porque se conecta a GND. Entonces sería FALLING si quiero que se active cuando lo tocan.

Para cuando lo sueltan = RISING.
Sin una resistencia que fije el nivel se puede quedar flotante. El pin lee cosas que nadie hizo.

GPIO 34 al 39 solamente entrada y no tienen pull-up. En el ejemplo del 35 se necesitaba una resistencia externa a 3,3V (10k).

Pines 6 al 11 no usarlos, son de la Flash del módulo. Los strapping se revisan cuando enciende el ESP32. Si se pone un nivel que no corresponde puede que no arranque.

21 para el botón, por ejemplo.
pinMode(21, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(21), isrBoton, FALLING);

Esto se coloca en setup una sola vez. El nombre isrBoton sin los paréntesis, porque aquí se indica qué función usar cuando llegue el aviso.

En la ISR no hacer cosas que se demoren. Ni Serial.println, ni delay. El watchdog puede reiniciar el ESP32 cuando se queda ocupado demasiado rato.

La ISR más bien deja una señal y sale. El resto se hace en loop, como prender el LED o mostrar algo en el monitor.

La Flash guarda el programa aunque apague el ESP32. SRAM para lo que se usa mientras funciona. IRAM es para instrucciones y DRAM para datos.

Caché: la CPU no tiene que ir a buscar todo a la Flash cada vez. Pero cuando el ESP32 escribe en Flash a veces se desactiva la caché un momento.

Si en ese momento ocurre una interrupción y la ISR estaba en Flash
IRAM_ATTR para que la función esté en la RAM de instrucciones. Igual tiene que ser seguro todo lo que use dentro, no solamente poner IRAM_ATTR.

volatile

El compilador puede tener una copia de una variable en un registro de la CPU. Si la ISR modifica el dato, el loop podría quedarse con el anterior.

volatile bool alerta = false;
La ISR pone alerta = true
En loop reviso esa bandera, la bajo y recién hago lo demás. No imprimir directamente desde la ISR.

La bandera no sirve para contar. Si pasan tres toques mientras está ocupado, queda un solo true. Para contar eventos se habló de contador o cola.

volatile no quiere decir que todo se haga de forma atómica, por ejemplo al sumar un contador o cuando el loop y la ISR lo cambian a la vez.

Rebote del pulsador

Aunque presione una vez, el contacto puede pasar varias veces de 1 a 0 muy rápido. Entonces la interrupción se ejecuta más de una vez.

Antirrebote. En clase se usó 50 ms.
Se compara cuánto pasó desde el toque anterior, pero hay que tener cuidado con la función de tiempo dentro de la ISR.

Botón → GPIO detecta el flanco → ISR → bandera. Ya en el loop se atiende.

Si quiero alarma apenas se presiona con pull-up: FALLING. Si no hay antirrebote, 10 pulsaciones no necesariamente producen 10 avisos.

En el repaso del timer: 1 MHz = 1 microsegundo cada tick. Con 500000 y autorrecarga da medio segundo, o sea 2 veces por segundo.